

App Development Project Report

App Dev Project Report

1. Student Details

Name: Sanidhya Srivastava

Roll Number: 24f2008062

Email: 24f2008062@ds.study.iitm.ac.in

About Me: I am a Diploma Level student at IIT Madras BS Degree Programme with deep interest in python backend and web development.

2. Project Details

Project Title: Trekking Management

Problem Statement:

To design and build a **Trekking Management Application** web application that allows **Admin, Trek Staff, and Users (Trekksers)** to interact with the system based on their roles.

Approach:

The app was built using flask for backend framework, and bootstrap for the frontend, using flask-sqlalchemy for the database management and jinja templating.

3. AI/LLM Declaration

I used Anitgravity agent to assist in writing Jinja templating, error understand and documentation

The extent of AI/LLM usage is around 10-15%, limited to code suggestions and documentation.

4. Technologies and Frameworks Used

Technology / Library	Purpose
Flask	Core backend web framework
SQLAlchemy	Object Relational Mapper for SQLite database
Jinja2	Template engine for rendering dynamic HTML pages
Bootstrap 5	Frontend styling and responsive design
SQLite	Lightweight local database for storing user data
Bcrypt	For password hashing

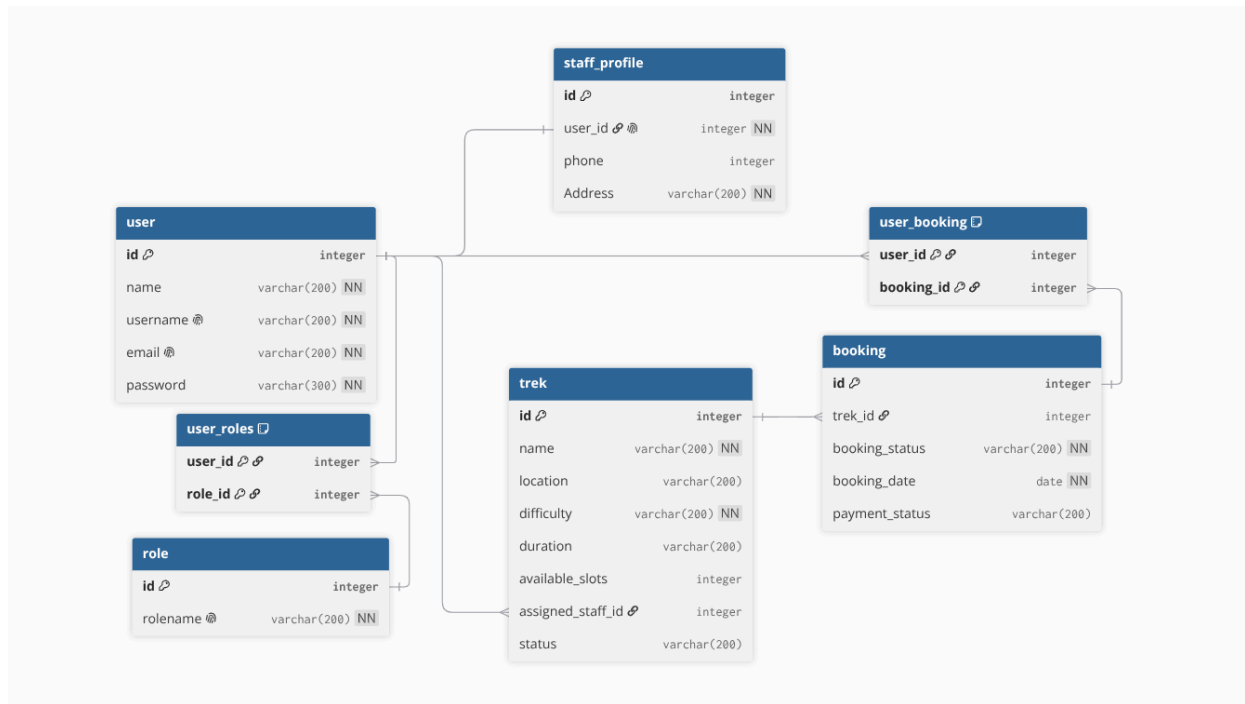
5. Database Schema / ER Diagram

Tables:

1. **User** — stores user account details (id, name, username, email, password)
2. **Role** — stores role definitions for access control (id, rolename)
3. **user_roles** — association table mapping users to their roles (user_id, role_id)
4. **staff_profile** — stores extended profile information for staff members (id, user_id, phone, Address)
5. **trek** — stores trek listings and management details (id, name, location, difficulty, duration, available_slots, assigned_staff_id, status)
6. **Booking** — stores trek booking records and status (id, trek_id, booking_status, booking_date, payment_status)
7. **user_booking** — association table mapping users to their bookings (user_id, booking_id)

Relationships:

- One-to-One → User → staff_profile
- One-to-Many → User (Staff) → trek
- One-to-Many → trek → Booking
- Many-to-Many → User ↔ Role (via user_roles)
- Many-to-Many → User ↔ Booking (via user_booking)



6. Application Routes Endpoints

Endpoint	Method	Description
<code>/login</code>	POST	Authenticate user credentials and generate active session
<code>/register</code>	POST	Register a new user account (Trekker or Pending Staff)
<code>/logout</code>	GET	Terminate user session and log out
<code>/complete-staff-profile/<user_id></code>	POST	Submit contact phone and address for newly registered staff
<code>/admin-dashboard</code>	GET	Fetch administrative dashboard data (treks, staff, users, bookings)
<code>/add-trek</code>	POST	Create a new trek entry with difficulty, duration, slots, and guide
<code>/edit-trek/<trek_id></code>	POST	Update trek information, available slots, and active status

<code>/delete-trek/<trek_id></code>	POST	Delete a trek entry from the database
<code>/approve-staff/<user_id></code>	POST	Approve pending staff registration and assign staff role
<code>/delete-staff/<staff_id></code>	POST	Remove staff user account and associated profile
<code>/blacklist/<user_id></code>	POST	Admin revokes access and blacklists a user
<code>/unblacklist/<user_id></code>	POST	Admin restores a blacklisted user to their previous role
<code>/assign-staff/<staff_id></code>	GET, POST	Admin assigns a staff member to lead a trek
<code>/search</code>	GET, POST	Admin searches across treks and users
<code>/staff-dashboard</code>	GET	Staff dashboard showing assigned treks and associated bookings
<code>/edit-staff-trek/<trek_id> ></code>	GET, POST	Assigned staff updates available slots and status of their trek
<code>/update-profile</code>	GET, POST	Staff updates their phone number and address
<code>/trekker-dashboard</code>	GET	Trekker portal showing available treks, active bookings, and history
<code>/trekker-search</code>	GET, POST	Trekker searches and filters treks by name, location, and difficulty
<code>/book-trek/<trek_id></code>	GET, POST	Trekker books a slot on an open trek
<code>/cancel-booking/<booking_id></code>	POST	Trekker cancels a booking and frees up the slot
<code>/update-trekker-profile</code>	GET, POST	

7. Architecture and Features (optional)

Architecture Overview:

- **app.py** – Main Flask application entry point, route controllers, session handling, and access control decorators.
- **models.py** – Relational database schema and SQLAlchemy ORM models (User, Role, trek, Booking, staff_profile, and association tables).
- **migrations/** – Flask-Migrate database version control and schema migrations.
- **/templates** – Server-side rendered Jinja2 HTML templates with layout inheritance (base.html).
- **/static** – Custom stylesheet (css/style.css) and static assets.
- **instance/app.db** – SQLite database storage.

Implemented Features:

- **User Registration & Login:** Supports separate roles for Admin, Staff, and Trekker.
- **Password Hashing:** Uses bcrypt to securely hash and store passwords.
- **Staff Approval System:** New staff registrations require Admin approval before they can access the staff dashboard.
- **Admin Dashboard:** Allows Admin to add, edit, and delete treks, assign guides, and manage users.
- **Staff Dashboard:** Allows staff members to view their assigned treks and see who has booked.
- **Trekker Dashboard:** Allows users to view available treks, book slots, and cancel bookings.
- **Search & Filter:** Allows trekkers to search treks by name, location, and difficulty level.
- **Booking & Slot Management:** Automatically updates remaining available slots when bookings are made or cancelled.
- **Trekking History:** Displays active bookings and completed treks separately.
- **Responsive UI:** Built using Bootstrap 5 for clean layout and tables.

Additional Features:

- **Blacklist User System:** Admin can blacklist problematic users to block their access, and unblacklist them later.
- **Profile Management:** Staff and Trekkers can update their contact details and profile information.

8. Video Presentation

Drive Link:

<https://drive.google.com/file/d/1sgTkBbRmZs9K9svGpXkipVxhG6XtUXmc/view?usp=sharing>

(Accessible to all with "View" permission.)
